

AI-Driven Testing and Code Review: Quality Assurance in the Agentic Era

Muhammad Swalha and Mohammed Abad

Course: Mass Production of AI Agents
Lecturer: Dr. Yoram Segal

Contents

1	Introduction — The Dawn of Agentic Quality Assurance	2
2	Defining Agentic QA — From Brittle Scripts to Autonomous Agents	2
3	Core Capabilities of Agentic AI Testing Systems	4
4	AI-Driven Code Review — Reimagining the Human Bottleneck	5
5	Market Landscape — Adoption, Investment, and Industry Momentum	7
6	Human-AI Collaboration and the Evolving Role of QA Professionals	8
7	Technical Architecture — Multi-Agent Systems in Software Testing	10
8	Challenges, Risks, and the Governance Imperative	11
9	The Road Ahead — Quality Assurance in a Fully Agentic World	12
10	Hebrew Summary	14
11	Data Visualizations	14
12	Block Diagram	14
	References	16

1 Introduction — The Dawn of Agentic Quality Assurance

Software quality assurance has always been a discipline caught between two competing pressures: the need to be thorough enough to catch defects before they reach users, and the need to be fast enough to keep pace with an accelerating development lifecycle. For decades, practitioners managed these pressures through a combination of manual testing, scripted automation frameworks, and increasingly sophisticated continuous integration pipelines. The introduction of AI into development workflows has not merely added a new tool to this arsenal — it has fundamentally reframed the challenge. The question is no longer *how to automate more tests faster*, but whether autonomous AI agents can take over the entire quality assurance lifecycle — perceiving application state, planning strategies, executing tests, and adapting to results — without waiting for human direction.

This transformation is arriving precisely as the software development process itself is being reimagined. With AI coding assistants now embedded in the workflow of the majority of professional developers, the volume of code being produced is growing at a rate that traditional QA approaches cannot absorb. As reported in the Anthropic 2026 Agentic Coding Trends Report, developers now use AI in roughly 60% of their work — yet they report being able to fully delegate only 0–20% of tasks to these systems, meaning AI operates more as a “fast copilot” than a full autopilot [1]. This gap between AI’s productivity gains and the supervisory burden it creates is the central challenge that agentic QA attempts to address.

The concept of agentic AI in quality assurance represents a qualitative leap beyond what the industry has experienced before. Unlike test automation frameworks that execute pre-written scripts, or AI tools that suggest code completions, agentic QA systems operate with genuine autonomy: they perceive the state of an application, reason about risk, devise a testing strategy, generate and execute test cases, analyze failures, identify root causes, and repair their own test suites when the application changes [4]. This lifecycle autonomy is new, and its implications for QA teams, development organizations, and software quality itself are profound.

This article examines the full scope of the agentic QA transformation — its technical foundations, its emerging market structures, its implications for QA professionals, and the significant challenges and risks that must be navigated to realize its promise. Drawing on recent industry research, landmark academic work, and empirical market data, it traces the arc from the obsolescence of brittle scripted automation to the architecture of multi-agent testing systems capable of operating at the speed and scale demanded by modern software development. It also addresses the critical question of trust: whether organizations are ready to govern AI agents making consequential decisions about software quality, and what frameworks are needed to ensure that the agentic era delivers genuine quality rather than merely high-volume, low-signal test output.

2 Defining Agentic QA — From Brittle Scripts to Autonomous Agents

The history of software testing automation is, in many ways, a history of fragility. The first generation of UI test automation tools — exemplified by Selenium and its derivatives — promised to eliminate the tedium of manual regression testing by recording and replaying

user interactions with web applications. In practice, these tools created a different kind of burden: test scripts were tightly coupled to specific UI element identifiers, layout structures, and application state sequences. Every time a developer updated a button's CSS class, renamed a field, or restructured a form, dozens or hundreds of test scripts broke simultaneously. Maintaining these scripts became a full-time occupation that, for many teams, consumed more engineering effort than the testing itself was worth. The Selenium era produced automation that was wide in coverage but brittle in practice — a paradox that agentic AI is now positioned to resolve [3].

The shift to agentic QA is not simply an incremental improvement on scripted automation; it is a categorical change in how testing systems relate to the software they test. Traditional automation is *prescriptive*: a human engineer specifies exactly what steps to execute, in what order, checking what conditions. Agentic AI is *adaptive*: an autonomous system observes the current state of the application, reasons about what aspects are most important to test given known risks and recent changes, generates test strategies accordingly, executes those tests, and updates its own understanding of the application as it encounters results [4]. This adaptiveness is what makes agentic AI capable of surviving — and thriving — in the kind of rapidly evolving codebases that modern development teams produce.

Agentic QA encompasses the full testing lifecycle in ways that no previous automation paradigm has managed. According to VirtuosoQA's analysis of the emerging landscape, agentic systems now handle autonomous test strategy creation, test case generation, execution, defect detection, root cause analysis, and self-healing test maintenance as an integrated, continuous process rather than a sequence of discrete manual handoffs [12]. Each of these capabilities, taken individually, represents a significant improvement over traditional automation; taken together, they constitute a fundamentally different approach to quality assurance — one in which the QA professional's role shifts from executing tests to supervising and directing a system capable of executing tests autonomously.

It is important to be precise about what "agentic" means in this context, because the term is used with varying degrees of rigor across the industry. At its weakest, "agentic" is simply a marketing term for AI-assisted tools that automate specific testing tasks. At its strongest, it refers to AI systems that implement a full perceive-plan-act-evaluate loop, maintaining persistent memory of application state and test history, using tools (browsers, APIs, databases) autonomously, coordinating with other specialized agents, and pursuing long-horizon testing goals without step-by-step human direction. The Atos framework usefully distinguishes three maturity levels: AI-assisted (Level 1), AI-augmented (Level 2), and AI-autonomous (Level 3), with most enterprises currently operating at Levels 1 or 2 and aspiring toward Level 3 [18]. This spectrum is a useful corrective to both excessive hype and unwarranted skepticism: agentic QA is real and already transforming organizations at Levels 1 and 2, while the full realization of Level 3 autonomy remains a work in progress.

The practical implications of this shift are already visible in industry data. According to ThinkSys's QA Trends Report 2026, 50.6% of QA teams now use AI for test-data creation, and 46% use AI for test-case formulation — metrics that would have seemed implausible to most practitioners even three years ago [16]. The scriptless testing paradigm, enabled by AI, is moving from experimental to standard practice, allowing QA teams to generate and maintain test suites at a scale that manual authoring could never support. The transition is uneven and incomplete, but its direction is unmistakable: the era of brittle scripts is ending, and the era of autonomous agents is beginning.

3 Core Capabilities of Agentic AI Testing Systems

The most transformative capability of agentic AI testing systems is arguably **self-healing**: the ability to automatically detect when application changes have broken a test script and to repair that script without human intervention. In the Selenium era, a single UI refactoring could invalidate hundreds of tests and require days of manual remediation. Agentic AI systems address this by maintaining a dynamic model of the application under test — tracking element relationships, interaction patterns, and expected behaviors — and automatically updating test scripts when the application changes in ways that affect them [3]. This capability alone can dramatically reduce the cost of test maintenance, historically one of the primary barriers to sustained investment in test automation. The Atos analysis identifies self-healing as one of the most transformative features of the agentic paradigm, noting its potential to shift QA teams from reactive firefighting to proactive quality strategy [18].

Autonomous test case generation from requirements represents another foundational capability. Rather than requiring QA engineers to manually translate user stories, acceptance criteria, and functional specifications into test scripts, agentic AI systems can ingest these artifacts directly and generate comprehensive test cases covering both happy paths and edge cases. This capability addresses a longstanding bottleneck: the gap between when requirements are written and when tests are available to validate them. By automating this translation, agentic systems allow testing to begin earlier in the development cycle, aligning with shift-left testing principles and enabling continuous validation of requirements as they evolve [3].

Root cause analysis — the ability to not merely detect test failures but to explain them — distinguishes agentic testing systems from earlier generations of automation. Traditional test runners report pass/fail results and stack traces; agentic systems analyze logs, trace execution paths through the codebase, correlate failures with recent code changes, and produce structured explanations of what went wrong and why [3]. This transforms the value proposition of automated testing: instead of generating a list of failures that engineers must manually investigate, agentic systems generate a prioritized list of defects with explanatory context, enabling faster triage and resolution. The Tricentis agentic QA framework explicitly identifies root-cause context as a core output of modern agentic testing pipelines, situating it alongside test generation and execution as an integrated capability [4].

Dynamic test data generation addresses one of the most persistent practical challenges in automated testing: the availability of appropriate, diverse, and realistic test data. Generating test data manually is time-consuming; using production data raises privacy and compliance concerns; static synthetic datasets quickly become stale and fail to cover emerging edge cases. Agentic AI systems generate contextually appropriate test data on demand, tailored to the specific requirements of each test scenario, including boundary conditions, negative cases, and representations of diverse user populations [4]. According to ThinkSys, more than half of QA teams have already adopted AI for test data creation, suggesting that this capability has crossed the threshold from experimental to mainstream [16].

Exploratory testing — the practice of investigating an application without a fixed script, following intuition and curiosity to discover unexpected behaviors — has traditionally been considered a uniquely human skill, resistant to automation. Agentic AI systems are beginning to challenge this assumption. Using browser automation frameworks like

Playwright integrated with Model Context Protocol (MCP) capabilities, agentic systems can navigate applications autonomously, probing for unexpected behaviors, testing combinations of inputs and states that scripted tests would never explore, and documenting discovered anomalies for human review [3]. This is not yet fully equivalent to the creative intuition of an experienced human tester — but it extends the reach of automated testing into territory previously inaccessible to machines.

The following table summarizes these core capabilities and their primary benefit to QA teams:

Capability	Traditional Automation	Agentic AI Approach	Primary Benefit
Test maintenance	Manual script repair after UI changes	Self-healing: automatic script repair	Reduced maintenance cost and downtime
Test case authoring	Manual translation of requirements	Autonomous generation from requirements	Faster coverage; earlier shift-left testing
Failure diagnosis	Stack traces and pass/fail flags	Root cause analysis with contextual explanation	Faster triage; actionable defect reports
Test data provision	Static datasets or manual generation	Dynamic, on-demand contextual data generation	Broader edge-case coverage; privacy compliance
Exploratory coverage	Human-only, time-bounded sessions	Autonomous agent-driven exploration via MCP	Wider anomaly discovery; continuous probing

The combination of self-healing, autonomous generation, root cause analysis, dynamic data generation, and exploratory testing gives agentic AI systems a comprehensive testing capability that represents a genuine step-change in what automated quality assurance can achieve.

4 AI-Driven Code Review — Reimagining the Human Bottleneck

Code review has long been recognized as one of the most valuable practices in software development — and one of the most expensive. The process of having experienced engineers read, evaluate, and provide feedback on each other’s code catches defects early, spreads knowledge across teams, enforces coding standards, and improves overall code quality. But it is also slow, inconsistent, and subject to reviewer availability and attention. In fast-moving development organizations, code review often becomes a bottleneck: pull requests queue up waiting for reviewer attention, context switches fragment concentration, and the pressure to keep pipelines moving leads to superficial reviews that miss important issues. AI-driven code review is emerging as a solution to this structural problem — not

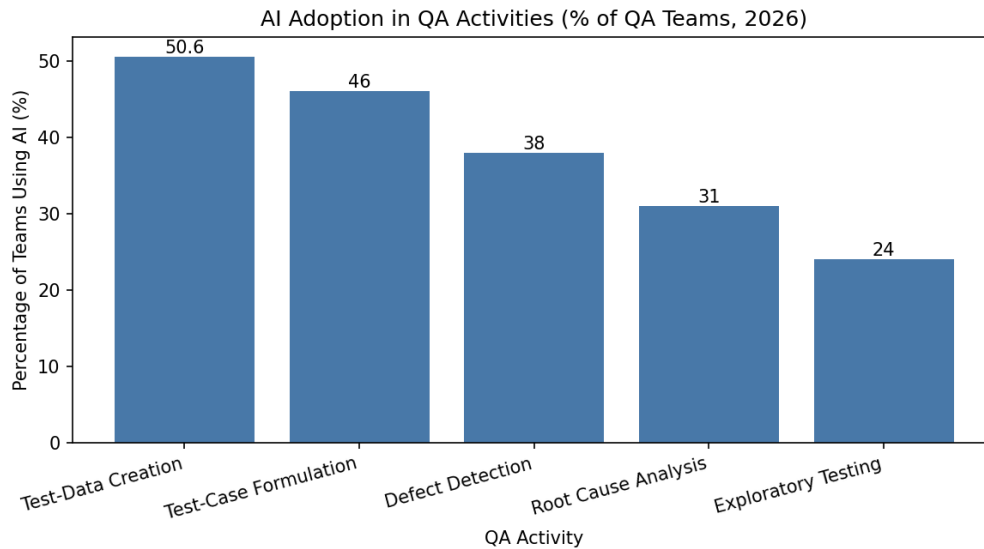


Figure 1: Bar chart comparing QA team adoption rates of five agentic AI testing capabilities (self-healing, test generation, root cause analysis, dynamic data, exploratory testing) versus traditional automation approaches

by eliminating human reviewers, but by automating the mechanical aspects of review and focusing human attention on the decisions that genuinely require human judgment.

A landmark academic contribution to this space comes from a paper accepted at ICSE 2026 (JAWs Workshop), titled *"Toward Agentic Code Review: Reimagining the Process in the AI Era."* The paper revisits the historical trajectory of code review, identifies persistent gaps in efficiency and traceability that have survived decades of tooling improvements, and proposes a framework through which generative and agentic AI can redefine code review — transforming it from a human bottleneck into a structured, automated process. In this framework, AI agents handle the systematic detection of defects, security vulnerabilities, style violations, and correctness issues, while human reviewers focus on architectural judgment, design decisions, and mentorship [19]. This academic framing is significant because it situates the current wave of AI code review tools within a longer history of practice, identifying what is genuinely new about the agentic approach and what remains to be solved.

The practical capabilities of current AI code review tools are already substantial. Leading platforms, as catalogued in the Augment Code survey of best AI code review tools for 2025, achieve bug detection rates of 42–48% in independent benchmarks — meaning that roughly half of all bugs present in reviewed code are automatically identified before a human reviewer sees the code [11]. These systems analyze code diffs automatically, flagging issues across multiple dimensions: logical bugs, null pointer dereferences, race conditions, security vulnerabilities including injection risks and authentication flaws, performance anti-patterns, and deviations from team coding standards. The breadth of this automated analysis is far greater than any individual human reviewer could reliably sustain across thousands of lines of code, and its speed — typically seconds to minutes rather than hours — allows it to integrate naturally into CI/CD pipelines without introducing delays.

The rise of AI-generated code creates a particularly acute need for AI-scale code review. As the Anthropic 2026 Agentic Coding Trends Report observes, organizations

are now deploying AI agents specifically to review the large-scale output of other AI agents, analyzing AI-written code for security issues, correctness, and style consistency — addressing the reality that AI-generated code now requires AI-scale review infrastructure [1]. This creates a new specialization within code review: agentic quality-control agents trained not just to detect general software defects but to recognize the specific failure modes of AI-generated code, including *hallucinated logic* (code that appears plausible but implements incorrect semantics), subtly wrong implementations that pass surface-level review but fail in edge cases, and security vulnerabilities that emerge from AI systems' incomplete understanding of security contexts.

The Applitools 2026 AI Testing Strategy analysis identifies a fundamental tension in AI-assisted development that has direct implications for code review: the biggest bottleneck is no longer test or review *volume*, but *signal-to-noise ratio* [9]. As AI tools generate more code faster, the volume of code requiring review increases proportionally — but if review tools respond to this volume by generating proportionally more review comments, the cognitive load on human engineers increases rather than decreases. The art of effective AI code review is not generating the maximum number of review comments but generating the most *meaningful* ones — filtering out trivial style issues, redundant checks, and low-confidence observations to surface only the findings that genuinely matter for software quality. This signal-focused philosophy is emerging as a design principle for the next generation of code review tools.

5 Market Landscape — Adoption, Investment, and Industry Momentum

The market data surrounding AI-driven testing and code review tells a story of exceptional momentum that, upon closer examination, reveals important nuances about the pace and distribution of adoption. The AI code review automation market was valued at \$1.8 billion in 2025 and is projected to reach \$9.4 billion by 2034, growing at a compound annual growth rate of 20.2% [8]. The broader AI code tools market, which encompasses both development assistance and testing tools, is projected to reach \$26.03 billion by 2030, growing at a CAGR of 27.1% from 2024 to 2030 [7]. These figures reflect genuine and sustained enterprise investment rather than speculative valuation — organizations are deploying AI testing and review tools in production environments, integrating them into CI/CD pipelines, and building operational workflows around their capabilities.

The projected market growth can be expressed in terms of compound annual growth rate using the standard CAGR formula:

$$\text{CAGR} = \left(\frac{V_{\text{final}}}{V_{\text{initial}}} \right)^{\frac{1}{t}} - 1 \quad (1)$$

where V_{final} is the projected market value at the end of the period, V_{initial} is the market value at the start of the period, and t is the number of years in the period. Applied to the AI code review automation segment — $V_{\text{initial}} = \$1.8\text{B}$ (2025), $V_{\text{final}} = \$9.4\text{B}$ (2034), $t = 9$ — this yields the reported 20.2% CAGR, confirming the consistency of the cited projections [8].

Gartner's predictions provide a useful macro-level frame for understanding where enterprise adoption stands and where it is heading. According to Gartner's analysis published in 2025, up to 40% of enterprise applications will include integrated task-specific

AI agents by the end of 2026, up from less than 5% in 2025 [14]. This represents an eight-fold increase in enterprise AI agent deployment within a single year — a growth rate that, if accurate, would make the agentic AI transition one of the fastest enterprise technology adoptions in history.

At the developer level, adoption of AI coding tools has reached near-ubiquity. According to the 2025 Stack Overflow Developer Survey, 84% of developers now use AI coding tools, and research cited by Digital Applied indicates that AI increases developer productivity by an average of 15–20%, though effectiveness depends heavily on task type and context [6] [5]. However, the sentiment data tells a more complicated story: positive developer sentiment for AI tools has decreased from over 70% in 2023–2024 to just 60% in 2025 [5]. This decline in enthusiasm despite continued high adoption rates suggests that developers are moving from the initial excitement of new capabilities into a more sober assessment of real-world tradeoffs — including the increased testing burden that AI-generated code creates, the inconsistency of AI suggestions, and the cognitive overhead of supervising AI agents.

The workforce transformation implications of agentic QA adoption are equally significant. VirtuosoQA reports that 90% of QA professionals who adopt AI collaboration are able to advance to strategic roles — a figure that reflects the upskilling opportunity that agentic tools create for practitioners willing to adapt [12]. QA engineers who previously spent the majority of their time authoring and maintaining test scripts are finding that AI tools can handle those tasks autonomously, freeing them for higher-value work in test strategy, risk assessment, and quality architecture. The critical caveat is that this transition requires deliberate investment in training and role redefinition — it does not happen automatically as a consequence of tool adoption alone.

The Tricentis QA Trends 2026 report identifies advanced agentic testing and new workforce training standards as the two most significant shifts in the current landscape, underscoring that the market transformation is not purely technical but organizational [15]. Organizations that treat agentic QA adoption as a tool procurement exercise, without investing in process redesign and skill development, are unlikely to realize the productivity gains the market data promises. This gap between the potential of agentic testing tools and the organizational readiness to deploy them effectively is one of the primary factors behind Gartner’s sobering counterpoint: over 40% of agentic AI projects will be canceled by end of 2027, citing escalating costs, unclear business value, or inadequate risk controls [13].

6 Human-AI Collaboration and the Evolving Role of QA Professionals

The emergence of agentic AI in software testing does not eliminate the need for human QA professionals — it profoundly transforms what those professionals are expected to do. The shift is from *execution to orchestration*: rather than writing test scripts, running test suites, and manually triaging failures, QA professionals in the agentic era are expected to evaluate AI agent output, provide strategic direction, make architectural decisions about testing coverage, and ensure that the overall quality assurance process is aligned with business risk priorities [1]. This is a fundamentally different job description, requiring different skills, different judgment, and a different relationship with the tools involved.

The Anthropic 2026 Agentic Coding Trends Report identifies eight major trends re-

shaping the software development lifecycle in the agentic era: SDLC transformation, multi-agent coordination, long-running agents, shifting engineering roles, agentic quality control, human-AI teaming, new workforce training standards, and the challenge of intent communication [1]. Each of these trends has direct implications for QA practice. Multi-agent coordination means that testing is no longer a single-stream process but a parallel, distributed activity involving specialized agents working simultaneously on different aspects of quality — performance, security, functional correctness, accessibility — with human QA engineers coordinating their outputs and resolving conflicts. Long-running agents introduce new requirements for test oversight: when an AI agent is running a multi-hour exploratory testing session, human engineers need tools to monitor its progress, intervene when it goes off course, and evaluate the quality of its findings.

The question of how much autonomy to grant AI testing agents is the central debate in the human-AI collaboration dimension of agentic QA. The Ministry of Testing forum discussions capture the range of practitioner opinion: some argue for AI systems that fully execute and even approve test cases autonomously, accelerating the QA process to match the speed of AI-assisted development; others advocate for human-in-the-loop approval gates for any high-risk testing decisions, including the approval of test coverage standards, the acceptance of defect severity classifications, and the sign-off on release readiness [17]. The right balance depends heavily on context — the risk profile of the application, the maturity of the AI testing system, the regulatory environment, and the organization’s tolerance for the kinds of errors that AI agents make differently from human testers.

Scriptless testing — where AI enables non-technical stakeholders to contribute to test creation without writing code — is broadening the definition of who “does QA” in an organization. When product managers, business analysts, and domain experts can describe testing requirements in natural language and have AI agents generate executable test cases from those descriptions, quality assurance becomes a distributed responsibility rather than the exclusive domain of a dedicated QA team [16]. This democratization of testing enables much broader test coverage, but also creates governance challenges: test suites authored by non-engineers may lack the technical depth needed to catch subtle defects, and the overall coherence of a testing strategy built from many distributed contributions requires careful coordination.

The training and skill development implications of this transformation are substantial. QA professionals who built their careers on deep expertise in specific automation frameworks — Selenium, Cypress, Appium — are finding that this expertise is declining in relative value as AI agents absorb the scripting work those frameworks required. The skills that are increasing in value are those AI cannot easily replicate: strategic thinking about risk and quality, domain knowledge about what software is supposed to do, judgment about when automated test results can be trusted, and the communication skills needed to translate quality concerns into business terms that organizational leaders can act on. Tricentis’s QA trends analysis identifies new workforce training standards as one of the two most significant shifts in the 2026 landscape, reflecting the industry’s recognition that tool adoption must be accompanied by deliberate skill development [15].

7 Technical Architecture — Multi-Agent Systems in Software Testing

The technical architecture of modern agentic testing systems reflects a fundamental insight: no single AI agent can effectively handle all aspects of the testing lifecycle simultaneously. Different testing tasks require different kinds of reasoning, different tool access, different memory structures, and different optimization criteria. Planning a testing strategy requires reasoning about risk and business priorities; generating test cases requires reasoning about code structure and boundary conditions; executing tests requires reliable, reproducible interactions with the application under test; analyzing failures requires cross-referencing multiple information sources; and repairing broken tests requires understanding the relationship between application changes and test preconditions. By decomposing these tasks into specialized agents that collaborate within a coordinated architecture, multi-agent systems achieve capabilities that no monolithic AI model could reliably provide [10].

The research paper *"Architecting Agentic AI for Modern Software Testing"*, published on ResearchGate, examines agentic AI impact across the full software testing lifecycle and proposes a scalable multi-agent system for automated test generation with core capabilities including dynamic test generation, tool use, memory, and planning [10]. This framework identifies the key architectural components that distinguish effective multi-agent testing systems from simpler AI testing tools:

- **Planning agents** that analyze application risk and define testing strategy
- **Generator agents** that create test scripts and test data
- **Execution agents** that run tests in realistic environments and monitor results
- **Repair agents** that maintain test suite health through self-healing mechanisms

The coordination of these agents requires a shared memory architecture — a persistent representation of application state, test history, known defects, and coverage metrics that all agents can read from and write to as they perform their specialized functions.

The integration of agentic testing systems with CI/CD pipelines represents a critical architectural challenge. Traditional automation testing slots cleanly into a CI/CD pipeline as a discrete step: run the tests, report results, gate the build. Agentic testing systems are more complex: they need to ingest code diffs and new requirements as pipeline inputs, generate targeted tests for the changed code, execute those tests in parallel with the full regression suite, analyze failures with root-cause context, and produce structured reports that support rapid decision-making [4]. Achieving this integration without introducing unacceptable latency — or creating noise that slows down rather than accelerates release decisions — requires careful system design. The trend is toward event-driven architectures in which agentic testing systems respond to code commit events asynchronously, producing continuous quality signals rather than blocking the pipeline.

The memory architecture of multi-agent testing systems deserves particular attention because it enables these systems to improve over time rather than starting fresh with each test run. Effective agentic testing systems maintain several distinct types of memory: *episodic* memory of specific test runs and their outcomes; *semantic* memory of the application's functional behavior and known defect patterns; *procedural* memory of testing strategies proven effective for specific types of code changes; and *contextual* memory

of the business priorities and risk tolerances that should guide test coverage decisions. The Applitools analysis of AI testing strategy emphasizes that teams must make intentional choices about where to apply AI testing tools, as indiscriminate use creates noise rather than confidence [9]. A well-designed memory architecture is what enables those intentional choices to be encoded as persistent system knowledge rather than requiring re-specification with each test cycle.

The challenge of tool use in agentic testing systems — how agents interact with browsers, APIs, databases, and other application components — is an active area of technical development. Frameworks like Playwright’s Model Context Protocol (MCP) integration are enabling AI agents to interact with web applications through the same interfaces that real users do, supporting the kind of realistic exploratory testing that scripted automation has always struggled to replicate [3]. Database agents can generate and manage test data within realistic schema constraints; API agents can construct complex sequences of API calls simulating real-world integration scenarios; security agents can probe for vulnerabilities using techniques borrowed from penetration testing. The convergence of these specialized tool-using agents within a coordinated multi-agent architecture is producing testing systems with a breadth and depth of coverage capability that represents a genuine step-change from previous generations of automation [10].

8 Challenges, Risks, and the Governance Imperative

Despite the genuine capabilities and market momentum of agentic QA, the path to widespread successful deployment is obstructed by a set of real and serious challenges. The most fundamental is the **trust gap**: organizations are adopting AI testing tools far faster than they are developing the governance frameworks needed to use them responsibly. According to the Tricentis 2026 QA Trends Report, only 35% of QA professionals feel ready to govern AI in QA — despite high adoption rates of the tools themselves [15]. This disconnect between tool adoption and governance readiness is the primary organizational risk in the current landscape. When AI testing agents make consequential decisions about test coverage, defect severity, or release readiness, and there is no clear accountability structure for reviewing those decisions, the organization is exposed to quality failures that may be harder to detect than those produced by traditional testing methods — precisely because they carry the appearance of automated validation.

The **non-determinism problem** in AI test generation raises technical challenges that do not have straightforward solutions. Unlike scripted test automation, where the same test produces the same result for the same application state (by design), AI-generated tests may vary in their structure, coverage, and sensitivity across runs, even for identical application states. This variability complicates test management in several ways: it makes it harder to track coverage over time, harder to reproduce test failures after a defect is fixed, and harder to maintain audit trails for compliance purposes [2]. Addressing non-determinism requires new validation frameworks that assess AI-generated tests not just for their immediate results but for their statistical properties across multiple runs — a significant additional engineering investment that many organizations have not yet made.

The **volume problem** created by AI-generated code is a systemic challenge that agentic testing must solve rather than merely absorb. As the Applitools analysis observes, AI-assisted development increases code volume and places proportionally greater pressure on QA teams, creating a feedback loop in which the same tools that accelerate development also accelerate the accumulation of testing debt [9]. The genuine solution

requires AI testing systems that are not just fast but *discriminating* — capable of identifying which tests provide meaningful quality signals and which are redundant, overlapping, or low-value. Building this discrimination into agentic testing systems requires sophisticated prioritization logic, empirical validation against historical defect data, and ongoing calibration by experienced QA engineers.

Security and correctness review of AI-generated code presents a specialized challenge. As the Anthropic 2026 report observes, agentic quality control agents must be trained to detect AI-specific failure modes that differ systematically from the kinds of bugs that human developers typically introduce [1]. AI-generated code may contain hallucinated logic — implementations that appear syntactically and semantically plausible but are functionally incorrect — as well as security vulnerabilities that emerge from the AI’s incomplete internalization of security best practices. Detecting these failure modes requires testing approaches that go beyond functional correctness checks to include adversarial probing, formal verification for critical components, and multi-agent review in which one AI agent specifically targets the output of another.

Gartner’s prediction that over 40% of agentic AI projects will be canceled by end of 2027 is not simply a forecast of failure — it is a diagnosis of the organizational conditions that lead to failure and a prescription for what needs to change [13]. Projects are canceled when costs exceed expectations, when business value is unclear, or when risk controls prove inadequate. All three failure modes are preventable with appropriate planning. Costs can be managed by starting with focused, high-value use cases rather than attempting full lifecycle automation immediately. Business value can be demonstrated by connecting testing metrics directly to business outcomes — defect escape rates, release cycle times, incident rates in production — rather than measuring only internal metrics. Risk controls can be built through the deliberate governance framework design that only 35% of organizations currently feel equipped to implement.

9 The Road Ahead — Quality Assurance in a Fully Agentic World

Imagining the near-term future of quality assurance requires holding two seemingly contradictory observations in mind simultaneously. On one hand, the technical trajectory of agentic AI testing is clear and rapid: systems are becoming more capable, more autonomous, more integrated, and more reliable at an accelerating pace. On the other hand, the organizational, ethical, and governance challenges of deploying these systems responsibly are significant and not reducible to technical solutions. The organizations that will define best practice in agentic QA will be those that advance on both dimensions in parallel — investing in technical capability while building the human infrastructure needed to direct, evaluate, and govern AI testing agents effectively.

The **convergence of agentic testing and agentic development** is perhaps the most important long-term trend to understand. As software development increasingly involves AI agents writing code, reviewing code, designing architectures, and managing deployment pipelines, quality assurance must evolve from its traditional position as an external check on developer output to become an integral component of a fully AI-mediated development process. This means testing agents that work alongside — and in continuous dialogue with — development agents, flagging quality issues in real time rather than batch-processing completed features, and providing the kind of immediate feedback that

allows the development process to self-correct before defects are deeply embedded in the codebase. The ICSE 2026 vision of agentic code review — turning code review from a human bottleneck into a structured, automated process — is the first step toward this integrated vision of AI-mediated quality [19].

The question of **intent communication**, identified in the Anthropic 2026 Agentic Coding Trends Report as one of the eight defining challenges of the agentic era, is particularly relevant to QA [1]. When an AI agent is executing a long-running testing session — exploring an application for hours, generating and running thousands of test cases — how does a human engineer communicate their intent clearly enough that the agent’s exploration is guided by the right priorities? Specifying “test the payment processing system” is insufficient; a human QA engineer would bring to that task years of domain knowledge, risk intuition, and business context not contained in the task description. Encoding this knowledge into the prompts, constraints, and evaluation criteria that guide agentic testing systems is the frontier challenge of the discipline — and it is fundamentally a communication and knowledge management challenge as much as a technical one.

The **workforce evolution** required by agentic QA is both an opportunity and a responsibility. VirtuosoQA’s observation that 90% of QA professionals who adopt AI collaboration advance to strategic roles [12] represents the opportunity: the automation of mechanical testing tasks creates space for QA professionals to do more strategically valuable work. But the responsibility is equally significant — this transition does not happen automatically, and organizations that fail to invest in training and role development will not achieve it. The QA professionals who will thrive in the agentic era are those who develop deep expertise in risk-based testing strategy, AI system evaluation, quality governance, and business-quality alignment — skills that require active cultivation and cannot be acquired simply by using AI tools.

Looking further ahead, the prospect of fully autonomous quality assurance — Atos’s Level 3 maturity, in which AI agents handle the complete testing lifecycle without human intervention for routine quality decisions — raises questions about accountability, liability, and the appropriate boundaries of machine autonomy in software quality contexts [18]. When AI agents make release readiness decisions for medical device software, financial systems, or critical infrastructure applications, the governance frameworks that ensure these decisions are made responsibly become matters of public safety, not merely organizational efficiency. The industry is still in the early stages of developing these frameworks, regulatory bodies are beginning to pay attention; and organizations that invest now in robust governance infrastructure will be better positioned to operate in the regulatory environment that is emerging.

The most durable insight from the current landscape may be this: *quality does not become less important because AI can generate it faster*. The purpose of quality assurance — ensuring that software does what it is supposed to do, safely and reliably — does not change when the tools used to achieve it become more powerful. What changes is the nature of the human contribution required: less script writing, more strategic judgment; less manual execution, more thoughtful governance; less reactive defect detection, more proactive quality architecture. The agentic era does not diminish the importance of QA professionals — it elevates it, demanding more sophisticated thinking about risk, value, and accountability from the people who will orchestrate the AI systems that are reshaping the discipline.

10 Hebrew Summary

סיכום בעברית — Hebrew Summary

פרק זה מסכם את עיקרי המאמר בשפה העברית לנוחות הקוראים דוברי העברית.

QA-מייצגות מהפכה של ממש בתחום ה (Agentic AI) בדיקות תוכנה בעידן הסוכנים האוטונומיים. AI בעוד שבעבר צוותי בדיקות הסתמכו על סקריפטים שבירים וכלי אוטומציה מסורתיים, כיום מערכות אוטונומיות מסוגלות לתכנן, לבצע, לנתח ולתקן מבחנים ללא התערבות אנושית שוטפת.

The transition from brittle scripted automation to autonomous agentic systems is well underway [4].

שוק סקירת הקוד האוטומטית עמד — לבדיקות ולסקירת קוד צומח בקצב מרשים AI-שוק כלי ה 2034 מיליארד דולר עד 9.4-וצפוי להגיע ל 2025 מיליארד דולר בשנת 1.8 על

35% יחד עם זאת, קיים פער משמעותי בין אימוץ הכלים לבין הבשלות הארגונית לנהל אותם: רק [15] בתהליכי הבדיקה של AI של (governance) מרגישים מוכנים לממשל QA-מאנשי המקצוע בתחום ה

ולא כמבצעי AI כמנחים אסטרטגיים של סוכני — שישכילו להסתגל לתפקידם החדש QA אנשי. The organizations and practitioners that embrace this transformation thoughtfully will define the future of software quality.

11 Data Visualizations

Further charts generated for this article.

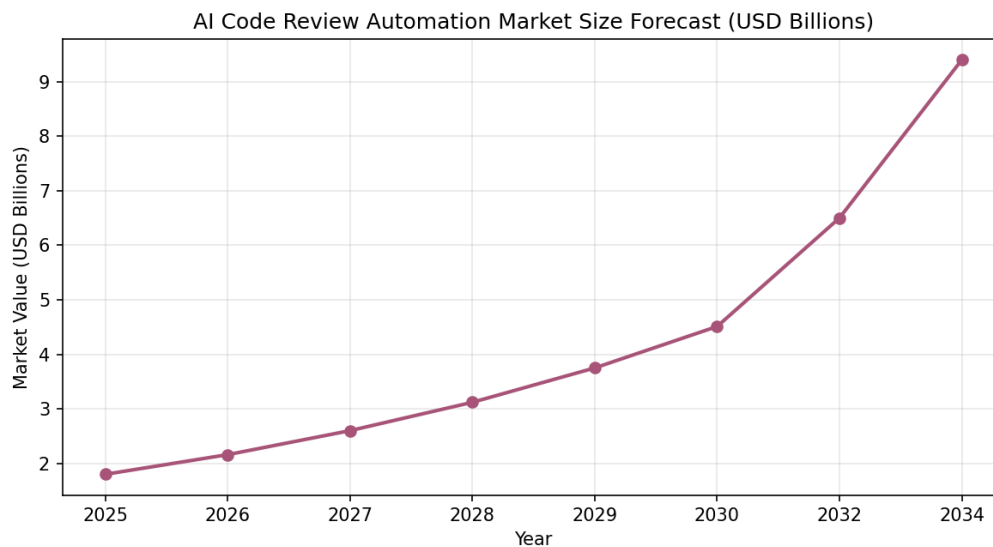


Figure 2: AI Code Review Automation Market Size Forecast (USD Billions)

12 Block Diagram

The diagram below shows a process described in this article.

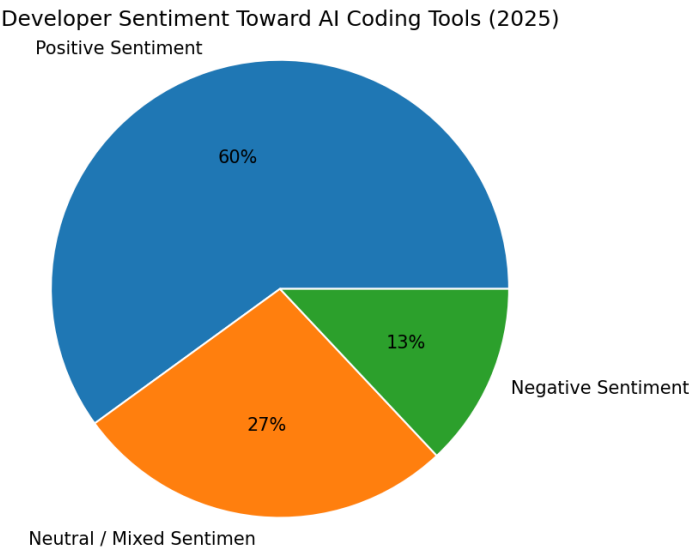


Figure 3: Developer Sentiment Toward AI Coding Tools (2025)

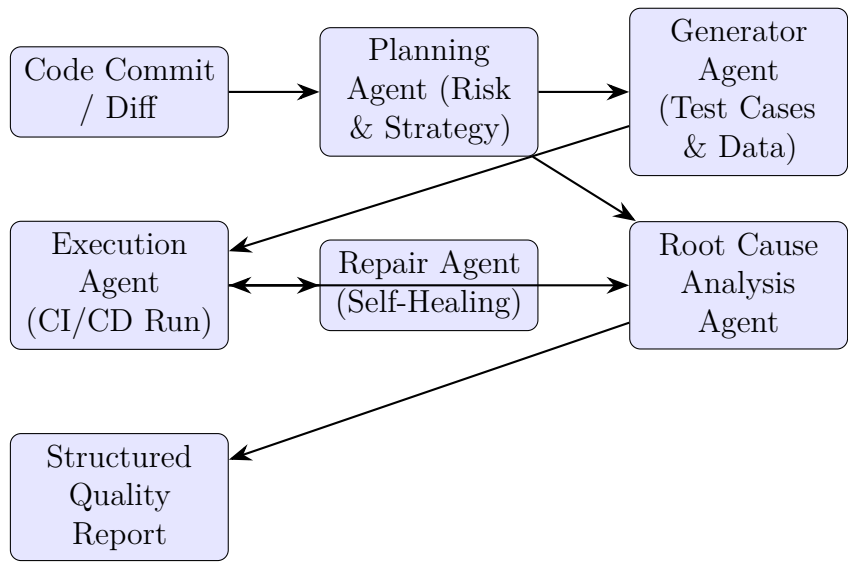


Figure 4: Multi-Agent Agentic QA Pipeline: from code commit to quality report

References

- [1] 2026 agentic coding trends report – anthropic. <https://resources.anthropic.com/2026-agentic-coding-trends-report>, 2026. Accessed 2026.
- [2] Agentic ai in software testing: The future of autonomous qa – aspiresys. <https://www.aspiresys.com/blog/software-testing-services/test-automation/embracing-agentic-ai-in-testing-autonomous-qa-and-the-future-of-test-automation/>, 2026. Accessed 2026.
- [3] Agentic ai: The shift to autonomous software testing – testquality. <https://testquality.com/agentic-ai-the-shift-to-autonomous-software-testing/>, 2026. Accessed 2026.
- [4] Agentic quality assurance: A guide to ai-driven qa – tricentis. <https://www.tricentis.com/learn/agentic-quality-assurance>, 2026. Accessed 2026.
- [5] Ai | 2025 stack overflow developer survey. <https://survey.stackoverflow.co/2025/ai>, 2026. Accessed 2026.
- [6] Ai code review automation: Complete guide 2025 – digital applied. <https://www.digitalapplied.com/blog/ai-code-review-automation-guide-2025>, 2026. Accessed 2026.
- [7] Ai code tools market size to reach \$26.03 billion by 2030 – grand view research. <https://www.grandviewresearch.com/press-release/global-ai-code-tools-market>, 2026. Accessed 2026.
- [8] Ai-generated code review tools market research report 2034 – dataintelo. <https://dataintelo.com/report/ai-generated-code-review-tools-market>, 2026. Accessed 2026.
- [9] Ai testing in 2026: Why signal, trust, and intentional choices matter – applitools. <https://applitools.com/blog/ai-testing-strategy-in-2026/>, 2026. Accessed 2026.
- [10] Architecting agentic ai for modern software testing – researchgate. https://www.researchgate.net/publication/392379751_Architecting_Agentic_AI_for_Modern_Software_Testing_Capabilities_Foundations_and_a_Proposed_Scalable_Multi-Agent_System_for_Automated_Test_Generation, 2026. Accessed 2026.
- [11] Best ai code review tools 2025 – augment code. <https://www.augmentcode.com/tools/best-ai-code-review-tools-2025>, 2026. Accessed 2026.
- [12] Future-proofing qa teams: Preparing for the agentic ai testing era – virtuosoqa. <https://www.virtuosoqa.com/post/future-proofing-qa-teams-preparing-for-the-agentic-ai-testing-era-virtuosoqa>, 2026. Accessed 2026.
- [13] Gartner: Over 40% of agentic ai projects will be canceled by end of 2027. <https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027>, 2026. Accessed 2026.

- [14] Gartner predicts 40% of enterprise apps will feature task-specific ai agents by 2026. <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>, 2026. Accessed 2026.
- [15] Qa trends for 2026: Ai, agents, and the future of testing – tricentis. <https://www.tricentis.com/blog/qa-trends-ai-agentic-testing>, 2026. Accessed 2026.
- [16] Qa trends report 2026 – thinksys. <https://thinksys.com/qa-testing/qa-trends-report-2026/>, 2026. Accessed 2026.
- [17] Rethinking qa strategy with next-gen ai & agentic ai – ministry of testing. <https://club.ministryoftesting.com/t/rethinking-qa-strategy-with-next-gen-ai-agentic-ai/86604>, 2026. Accessed 2026.
- [18] Testing in the age of agentic ai – atos. <https://atos.net/en/blog/testing-in-the-age-of-agentic-ai>, 2026. Accessed 2026.
- [19] Toward agentic code review: Reimagining the process in the ai era – icse 2026 / jaws. <https://conf.researchr.org/details/icse-2026/jaws-2026-papers/58/Toward-Agentic-Code-Review-Reimagining-the-Process-in-the-AI-Era>, 2026. Accessed 2026.